Constrained Decoding for Robust NL-to-SQL

Jeongsik Park, Haiyun Lyu, Hao-Tien Cheng, Manu Goel,
Wanting Jiang, Zihan Xie, Zhuoning Wu
University of Southern California
{jeongsik, haiyunly, haotienc, manugoel, wantingj, xiezihan, zhuoning}@usc.edu

Abstract

Large language models have improved natural language to SQL generation, but free-form decoding often produces invalid or non-executable queries. We study whether decoding-time structural control can improve reliability by evaluating Qwen3-0.6B in thinking and non-thinking modes under two constrained decoding methods, XGrammar and Outlines, on WikiSQL and Spider datasets. Constraints are designed on WikiSQL as an in-distribution testbed and then transferred to Spider as a generalization check. We measure execution accuracy, valid SQL rate, executable rate, and latency. Our results show an asymmetric effect: on the simpler WikiSQL distribution, constrained decoding (especially XGrammar) improves SQL validity and execution accuracy, while on Spider's multi-table schemas the same constraints transfer poorly and can degrade accuracy. We find that constrained decoding must be co-designed with the target distribution, and that reasoning-mode generation interacts non-trivially with grammar enforcement.

1 Introduction

Translating natural-language questions into executable SQL (NL2SQL) is a standard way to let non-experts query relational databases (Zhong et al., 2017; Yu et al., 2018b). Modern LLMs can produce fluent SQL, yet free-form decoding still yields outputs that fail to parse, reference nonexistent columns, or silently return the wrong result. These failures are costly in practice because every downstream component (the SQL engine, the database, the application layer) assumes a well-formed query.

Two broad remedies have been proposed. *Fine-tuning* adapts model parameters to SQL distributions (Scholak et al., 2021; Zheng et al., 2024; Hu et al., 2021), while *constrained decoding* enforces structural validity at inference time by masking tokens that would violate a grammar or regex (Scholak et al., 2021; Willard and Louf, 2023; Dong et al., 2025; Gat, 2024). Orthogonally, recent

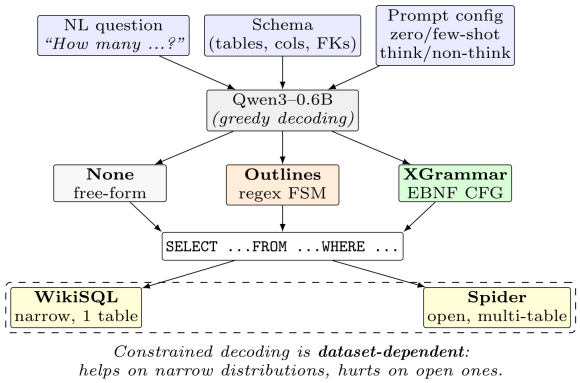


Figure 1: Overview of our NL2SQL framework.

open-weight models expose a *thinking mode* that emits an explicit chain-of-thought before the final answer (Wei et al., 2022; Qwen Team, 2025).

We study NL2SQL as a controlled experiment on decoding-time structural control. Using Qwen3-0.6B (Qwen Team, 2025) in both thinking and non-thinking modes, we evaluate two representative constrained decoders: XGrammar (Dong et al., 2025) (context-free grammar) and Outlines (dottxt-ai, 2025; Willard and Louf, 2023) (regular-expression FSM) on WikiSQL (Zhong et al., 2017) and Spider 1.0 (Yu et al., 2018b). We design the grammars on WikiSQL first, where the query space is narrow and can be described exhaustively, and then transfer them to Spider with only the minimum changes needed for joins, ordering, and multi-table schemas. This mirrors a realistic deployment scenario in which practitioners calibrate a decoder on one benchmark and reuse it elsewhere. LM Format Enforcer (Gat, 2024), was evaluated in our midterm but excluded from the final sweep on cost grounds (§4.5). We ask three questions: (1) Does constrained decoding reduce invalid and non-executable SQL without training? (2) Does reasoning mode help, hurt, or interact with grammar enforcement? (3) Do constraints tuned for a narrow schema transfer to an open, multi-table one?

Our contributions are fourfold. (i) We release a clean experimental pipeline that holds the

model, prompt, and decoder fixed while varying only the constraint, dataset, and reasoning mode. (ii) We evaluate across four complementary metrics: valid SQL rate, executable rate, execution accuracy, and latency with paired error analysis over 24 experiments. (iii) We provide evidence that constrained decoding is *dataset-dependent*: XGrammar matches or exceeds the unconstrained baseline on WikiSQL, but both constrained decoders lose on Spider, suggesting that grammars must be co-designed with the target distribution rather than reused as-is. (iv) We show that a prompt-and-grammar design that admits a `<think>. . . </think>` prefix inside the regex/EBNF keeps thinking mode compatible with constrained decoding, reversing the negative interaction observed in our earlier unconstrained-thinking baseline. Code and evaluation outputs are released at `https://github.com/lucasjeongsikpark/CSCI566-GroupProject`.

2 Related Work

2.1 NL2SQL benchmarks

WikiSQL (Zhong et al., 2017) popularized single-table `SELECT–WHERE` generation with a large set of crowd-sourced questions, and remains a standard testbed for evaluating whether a model can pick the right column, aggregation, and filter. Spider 1.0 (Yu et al., 2018b) raised the bar with cross-domain generalization to unseen databases, multi-table joins, and more compositional queries. Subsequent benchmarks extend this setting to conversations (Yu et al., 2019) and to enterprise-scale workflows (Lei et al., 2024). We stay with WikiSQL and Spider 1.0 because they isolate decoding-time effects without introducing agent-level confounds.

2.2 LLMs for SQL generation

Early neural NL2SQL systems used schema-aware encoders (Xu et al., 2017; Yu et al., 2018a; Wang et al., 2020), whereas modern approaches either prompt or fine-tune LLMs directly (Zheng et al., 2024; Hu et al., 2021). Chain-of-thought and reasoning-style prompting can improve semantic correctness on complex queries (Wei et al., 2022; Kojima et al., 2022), and the current generation of open-weight models, including Qwen3 (Qwen Team, 2025), exposes an explicit *thinking mode* through the chat template. We exploit this reasoning behavior through the switchable thinking flag rather than through fine-tuning, and rely on retrieval-based in-context demonstrations (Liu et al., 2022; Brown et al., 2020) for task-specific context.

2.3 Constrained decoding

Constrained decoding restricts the output to a predefined language by masking tokens that cannot extend into a valid string. PICARD (Scholak et al., 2021) applied incremental parsing to SQL decoding and showed that it raises execution accuracy. More recently, Outlines (dottxt-ai, 2025) compiles regular languages into tokenizer-aligned finite-state automata for efficient masking (Willard and Louf, 2023), while XGrammar (Dong et al., 2025) compiles context-free grammars into a stack-based parser with near-zero per-token overhead. LM Format Enforcer (Gat, 2024) occupies the same design space via pattern-based filtering, but in our setup it is an order of magnitude slower per decoding step than either Outlines or XGrammar (§4.5), so we exclude it from the final sweep. Compared with prior work, our contribution is not a new decoder but a controlled, multi-metric comparison between a CFG decoder and its regex projection on the *same* task, under both reasoning modes, and across two datasets chosen to probe in-distribution versus transferred behavior.

3 Datasets

Table 1 summarizes dataset sizes, and Table 2 shows representative examples. Because our hardware budget (§4.3) rules out running inference over the full test sets, we sample 300 examples from WikiSQL and 150 from Spider using a fixed seed (42), and reuse the same subsets across all 24 decoding conditions so that every comparison is strictly paired.

3.1 WikiSQL

WikiSQL (Zhong et al., 2017) is a large-scale single-table NL2SQL benchmark whose queries follow a `SELECT`–aggregation–`WHERE` template. The restricted structure and limited schema variability make it a controlled environment for designing and tuning constrained decoders. The grammar and regex we build in §4.5 cover essentially the entire query space of the benchmark. Therefore, we use WikiSQL as our primary *in-distribution* testbed for measuring whether constraints improve syntactic validity and execution correctness, and we standardize the table identifier to a single canonical name `table` so the model only has to pick columns.

Dataset	Train	Val	Test	Query type	Schema complexity
WikiSQL	56,355	8,421	15,878	Single-table SELECT	Narrow (1 table/query)
Spider 1.0	7,000	1,034	2,147	Multi-table, joins	Open (multi-table, FK joins)

Table 1: Statistics and characteristics of datasets.

Dataset	Question (Input)	SQL Query (Output)
WikiSQL	How many schools did player number 3 attend?	`SELECT COUNT(School) FROM table WHERE Player = 3`
Spider	Find the top-selling product among customers who bought 'Youtube Men's Vintage Henley' in July 2017, excluding itself.	`SELECT product_name FROM sales WHERE product_name != 'Youtube Men"s Vintage Henley' AND date LIKE '2017-07%' ORDER BY units_sold DESC LIMIT 1`

Table 2: Representative NL question / SQL query pairs.

3.2 Spider 1.0

Spider (Yu et al., 2018b) is a cross-domain, multi-table benchmark that requires generalization to unseen schemas, including `JOIN` operations, subqueries, and compositional structures. We use Spider as a *generalization check* for constrained decoders originally designed on WikiSQL: the same grammar skeleton is reused, and we add only the minimum extensions needed (multi-table `FROM`, `JOIN . . . ON`, `ORDER BY`, `LIMIT`), without re-tuning per database. Spider 2.0 (Lei et al., 2024) targets agent-based enterprise workflows and therefore introduces system-level confounds orthogonal to decoding, so we adopt Spider 1.0 as a sufficiently challenging but model-only benchmark.

4 Method

This section frames NL2SQL as a controlled study of constrained decoding. We outline the design (§4.1), then cover data preprocessing (§4.2), the model and hardware (§4.3), prompt construction (§4.4), the constrained decoders we evaluate (§4.5), and the evaluation protocol (§4.6).

4.1 Design

Our central hypothesis is that constrained decoding is *dataset-dependent*: a grammar that stabilizes one distribution may transfer poorly to another. To test this, we design both decoders on WIKISQL (Zhong et al., 2017) as an in-distribution testbed, and then transfer them to SPIDER (Yu et al., 2018b) with the minimum changes required for multi-table queries. We neither re-tune per database nor adapt model parameters.

4.2 Data preprocessing

Both datasets are converted into a unified JSON schema with three fields: *question*, serialized *schema*, and *gold SQL*. For WikiSQL we reconstruct canonical `SELECT` + agg + `WHERE` queries from the structured annotations using the official operator tables, and we rewrite every example with a single canonical table name `table` so the model only has to pick columns. For Spider we join each example with its database schema entry (`table_names_original`, `column_names_original`, `column_types`, `primary_keys`, `foreign_keys`) and validate that the referenced `db_id` exists and the gold SQL is present. Violations are logged and excluded.

4.3 Model and hardware

Model choice. We use Qwen3-0.6B (Qwen Team, 2025), the smallest text-only model in the Qwen3 family. We avoid the Qwen-VL vision variants (Bai et al., 2023) because our task is text-only, and we use Qwen3 rather than earlier Qwen2.5 releases (Yang et al., 2024) because Qwen3 natively exposes a switchable `enable_thinking` flag in its chat template. When enabled, the model emits a `<think>. . . </think>` trace before the SQL, and when disabled, it skips the trace.

Why the smallest variant. All experiments run on a single Apple M1 Pro laptop with 32 GB unified memory, using the PyTorch `mps` backend. Latency is measured at the batch level (see §5.4). Under this accounting, a single thinking-mode condition on WikiSQL already takes roughly 0.8 h to 3.0 h of wall time, and a thinking-mode condition on Spider takes roughly 0.8 h to 2.2 h. Summed over our 24 conditions, the full experimental sweep consumes about 33 hours of wall time on this device. Larger Qwen3 variants (1.7 B, 4 B, 8 B) would push this well beyond our feasible budget.

Few-shot learning. The same hardware constraint rules out practical fine-tuning. Even parameter-efficient adaptation of a reasoning model is hard to fit on a 32 GB M1 Pro, and neither WikiSQL nor Spider ships with ground-truth `<think>` traces, so supervised fine-tuning of the reasoning behavior would additionally require synthesizing traces with a larger teacher model. We therefore exploit the large unused training splits of both

benchmarks through in-context learning instead. Few-shot prompting puts these training examples to work without any gradient update, and the brittleness of zero-shot few-shot differences is well documented for smaller LLMs (Brown et al., 2020). In our main runs we use $N{=}1$ demonstration retrieved from the same dataset's training split (retrieval details in §4.4), and we also report a zero-shot baseline for comparison. This setup isolates constrained decoding as the only intervention that modifies generation itself, while still letting the model benefit from task-specific context.

Decoding hyper-parameters. All conditions use deterministic greedy decoding (`do_sample=False`), so `temperature`, `top_p`, and `top_k` are not applied and no `repetition_penalty` is set. We leave `num_beams` at its default of 1, so no beam search is used. The token budget is `max_new_tokens=150` in non-thinking mode (which comfortably covers a single SQL query) and `max_new_tokens=1,500` in thinking mode (which additionally has to fit the `<think>` trace). Tokenization uses left padding with `pad_token` falling back to `eos_token` for batched generation.

4.4 Prompt construction

Prompts use the Qwen chat template with a *system* message describing the role and dataset-specific rules, and a *user* message containing the per-instance schema and question. The Spider system message additionally lists nine rules (SQL must start with `SELECT`, use only schema elements, join only through listed foreign keys, qualify ambiguous columns, preserve string literals exactly, and so on) because Spider's open schema tolerates many structurally valid but schema-invalid outputs. A worked example of the full WikiSQL and Spider prompts (system + user + few-shot block) is given in Appendix B.

Few-shot retrieval. In the few-shot regime we prepend N demonstrations ($N{=}1$ in our main runs) retrieved from the same dataset's training split. Rather than sampling shots uniformly at random, we rank the training questions by TF-IDF cosine similarity (1–2-grams, `TfidfVectorizer`) to the test question and return the top-k (Liu et al., 2022). For WikiSQL each shot carries its single-table schema. For Spider we strip schemas from shots because per-database schemas differ from the target database and would otherwise bloat the prompt.

4.5 Constrained decoding

Constrained decoding masks, at every step, tokens that would extend the partial output outside a predefined language (Scholak et al., 2021; Willard and Louf, 2023). We evaluate two representative implementations: **XGrammar** (Dong et al., 2025), a context-free grammar (CFG) decoder, and **Outlines** (dottxt-ai, 2025; Willard and Louf, 2023), a regular-expression FSM decoder.

4.5.1 XGrammar (EBNF, per-example).

For each example we build an EBNF from its schema, with dataset-specific switches. The `SELECT` clause admits a column, an aggregation (`COUNT/MAX/MIN/SUM/AVG`), or `*`. The `WHERE` clause is emitted as an explicit set of alternatives covering 0 to K conditions ($K{=}4$). An explicit enumeration avoids the ambiguity of `(. . . )*` repetitions, which in practice caused XGrammar to admit infinite condition-repetition loops that surfaced as overgeneration. Operators include {`=`, `!=`, `>`, `<`, `>=`, `<=`, `LIKE`, `IN`, `NOT IN`, `IS NULL`, `IS NOT NULL`}. Column and table terminals are listed as literal alternatives drawn from the current schema. For Spider we additionally enable `JOIN. . . ON`, `ORDER BY`, and `LIMIT` (bounded to 6 digits), and allow `OR` in addition to `AND`. For WikiSQL we restrict to a single `table` identifier, no joins, and `AND` only.

4.5.2 Outlines (regex, dataset-level).

For WikiSQL a single dataset-wide regex covers the entire query space. For Spider the regex is parameterized by the current database: the union of its table names is inlined as a character class, and the regex admits up to three `JOIN` clauses, up to four `WHERE` conditions, an optional `ORDER BY`, and a bounded `LIMIT`. We invoke Outlines via `OutlinesCoreBackend`'s `regex_logits_processor` and plug it into `model.generate` directly, reusing the unconstrained baseline's code path.

Why not LM Format Enforcer. A natural third candidate is LM Format Enforcer (LMFE) (Gat, 2024), which enforces regex or JSON-schema constraints by filtering the logit distribution against a character-level pattern matcher. We benchmarked LMFE in our midterm report but dropped it from the final evaluation on cost grounds: under our setup LMFE averages roughly 12 minutes per example, so the 5,400-example sweep (24 conditions $\times$ 150–300 examples) would have exceeded our

hardware budget by an order of magnitude. The slowness stems from LMFE scoring every vocabulary token against the running pattern in Python at each decoding step, rather than pre-compiling the constraint into a tokenizer-aligned automaton the way Outlines and XGrammar do.

Design flow. We designed the admissible language *first* as an XGrammar EBNF, because a CFG is the most expressive of the two formalisms and a natural fit for SQL's nested clause structure. We then *projected* this grammar down to an equivalent regular expression for Outlines. Because regular languages cannot express everything a CFG can, this projection loses some structural guarantees. For example, the Outlines regex bounds `WHERE` and `JOIN` counts explicitly rather than recursively. As we show in the results, this projection gap partly explains why Outlines trails XGrammar across most settings. We list the final grammars in Appendix A.

Supporting thinking mode under constraints. In our earlier unconstrained runs, thinking mode hurt SQL quality, largely because the `<think>` trace leaked tokens into the final answer and the decoder had no clear boundary for where the answer began. To address this, we make constrained decoding compatible with thinking mode by allowing an optional `<think>. . . </think>` *prefix* inside the admissible language. Specifically, for Outlines, we prepend a FSM-safe fragment `(<think>([^<]|<[^/])*</think>\s*)?`, avoiding lookaheads since `interegular` does not support them. For XGrammar, we extend the root rule to `root ::= think_block select_stmt | select_stmt`, with `think_text ::= ([^<] | "<" [^/])*`, which is the CFG analogue of the regex.

In both cases, the decoder may emit a reasoning trace, but the grammar enforces that whatever follows `</think>` is a valid SQL query. This change makes thinking mode competitive with non-thinking mode, whereas the previous report observed the opposite.

Post-processing. Before evaluation we strip everything up to `</think>` (or fall back to the last `select` if the tag is missing), remove Qwen control markers and Markdown fences, truncate at the first semicolon, and reject queries with unbalanced parentheses, trailing operators, or no `SELECT`. Rejected outputs become empty strings and count as invalid. Concrete examples of each rule are shown in Appendix C.

4.6 Evaluation

We report four metrics per condition, all computed post-hoc from a single inference run.

Valid SQL Rate. The fraction of predictions that parse, measuring syntactic well-formedness independent of schema grounding or semantics.

Executable Rate. The fraction of predictions that both parse and execute without runtime errors on the target database. The gap to Valid SQL Rate reflects schema-grounding errors that grammar constraints cannot capture (e.g., referencing non-existent columns despite syntactic validity).

Execution Accuracy. The fraction of predictions whose execution result (compared as a sorted multiset of stringified rows) matches that of the gold query. This is our primary end-to-end metric and follows the Spider evaluation protocol (Yu et al., 2018b; Zhong et al., 2017). For WikiSQL, we bind the canonical `FROM table` to `table_<db_id>` in a shared SQLite, rewrite columns to `col<i>`, and apply `COLLATE NOCASE` to string literals. For Spider, each question is routed to its corresponding SQLite database via a `question→db_id` index. Examples whose gold query fails to execute are excluded.

Latency. We measure latency using `time.perf_counter()` around each `model.generate` call, normalized per example by batch size, and report it alongside token counts. Aggregates and tokens-per-second are reported in §5.4.

5 Results and Discussion

We report four complementary metrics: *valid SQL rate*, *executable rate* (ER), *execution accuracy* (EA), and *latency/throughput*. Valid SQL rate captures structural reliability, ER reflects successful execution, and EA measures end-to-end correctness via result equivalence. Latency and throughput quantify computational cost.

5.1 WikiSQL

Table 3 consolidates valid SQL rate, executable rate, and execution accuracy on the 300-example WikiSQL evaluation split. Three consistent patterns hold across all three metrics.

First, XGrammar is consistently the strongest decoder on WikiSQL. It achieves the highest valid SQL rate (up to 98.7%), the highest executable rate (up to 98.7%), and the highest or near-highest

	Valid SQL (%)			Executable (%)			Exec. Accuracy (%)		
	None	Outl.	XGram.	None	Outl.	XGram.	None	Outl.	XGram.
Zero-shot									
Non-thinking	93.7	78.0	**96.7**	93.7	78.0	**96.7**	39.3	36.0	**45.3**
Thinking	87.3	71.7	**89.3**	87.7	71.3	**89.7**	**55.0**	42.7	54.0
Few-shot									
Non-thinking	91.0	74.7	**98.7**	91.0	75.3	**98.7**	62.7	49.3	66.7
Thinking	90.0	63.7	**95.3**	90.0	77.0	**95.3**	66.3	50.0	**68.3**

Table 3: WikiSQL results (300 examples). Valid SQL rate, executable rate, and execution accuracy across all settings. Bold marks the best decoder per row within each metric. XGrammar leads or matches the unconstrained baseline on all three metrics, while Outlines consistently underperforms.

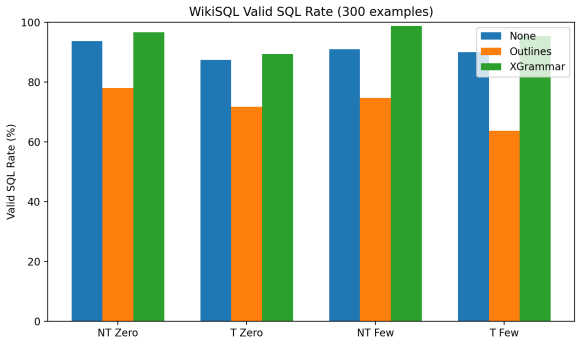


Figure 2: Valid SQL rate on WikiSQL (300 examples) across decoding methods. XGrammar gives the highest structural validity in both zero-shot and few-shot settings, while Outlines consistently underperforms.

execution accuracy (up to 68.3%) in every setting. Figure 2 visualizes the same trend for structural validity: XGrammar remains best in all four zero/few-shot and thinking/non-thinking settings. Outlines, by contrast, underperforms the unconstrained baseline by 12–16pp on validity and executability, and by 3–16pp on EA. The gap between the two constrained decoders reflects their underlying formalisms: XGrammar's context-free grammar can express SQL's nested clause structure natively, whereas Outlines' regular-expression FSM must approximate it with bounded repetition, losing structural guarantees in the process.

Second, few-shot prompting provides large and consistent gains on WikiSQL. EA improves by 11–23pp moving from zero-shot to few-shot across all settings (Figure 4, top row). The improvement is larger for non-thinking mode (+23pp for None, +21pp for XGrammar) than for thinking mode (+11pp, +14pp), suggesting that few-shot examples partially substitute for chain-of-thought reasoning on WikiSQL's simple queries.

Third, the thinking vs. non-thinking gap narrows under few-shot prompting. In zero-shot, thinking mode leads by 10–16pp on EA. In few-shot, the gap shrinks to 1–4pp, with XGrammar achieving nearly identical EA in both modes (66.7% vs. 68.3%). This convergence suggests that on WikiSQL's restricted query space, grammar constraints plus few-shot examples can largely compensate for the absence of explicit reasoning traces. This also validates the prompt-and-grammar design described in §4.5: by allowing a `<think>` prefix inside the grammar, thinking mode is no longer degraded by constrained decoding unlike in our earlier unconstrained-thinking baseline.

WikiSQL error analysis. The remaining invalid outputs under XGrammar are mostly syntax-related, with very few schema errors. Outlines produces substantially more schema errors and syntax failures; in the few-shot thinking setting it also generates a notable number of empty outputs. For the unconstrained baseline, invalid outputs split evenly between syntax errors and schema mismatches.

5.2 Spider

Table 4 consolidates the same three metrics on the 150-example Spider evaluation split. The picture is sharply different from WikiSQL.

First, constrained decoding severely degrades all three metrics on Spider. The unconstrained baseline achieves 70–83% on valid SQL, 74–83% on executability, and 22–36% on EA, while both Outlines and XGrammar collapse to 26–47% valid SQL, 34–60% executability, and 7–11% EA. The drop is 15–29pp on EA alone—far larger than on WikiSQL, where XGrammar was competitive with the baseline. This confirms our central hypothesis: grammars tuned on a narrow distribution (WikiSQL) do not transfer to a broader one (Spider) without re-design.

Second, few-shot prompting does not help on Spider—and in some cases hurts. Thinking mode with no constraints drops from 36.0% EA (zero-shot) to 28.0% (few-shot), an 8pp decrease. Constrained settings show negligible changes ($\pm$3pp). The likely explanation is that Spider's diverse schemas require compositional reasoning that a single retrieved demonstration cannot provide, and

	Valid SQL (%)			Executable (%)			Exec. Accuracy (%)		
	None	Outl.	XGram.	None	Outl.	XGram.	None	Outl.	XGram.
Zero-shot									
Non-thinking	76.0	34.0	32.7	**77.3**	48.0	34.0	**23.3**	8.0	6.7
Thinking	**82.7**	26.0	36.0	**82.7**	42.7	36.0	**36.0**	6.7	10.7
Few-shot									
Non-thinking	**77.3**	47.3	36.7	**77.3**	60.0	36.7	**22.0**	8.0	9.3
Thinking	**70.7**	30.7	32.7	**74.0**	49.3	36.0	**28.0**	10.0	9.3

Table 4: Spider results (150 examples). Unconstrained decoding dominates all three metrics in every setting. Both constrained decoders substantially degrade validity, executability, and accuracy, confirming that grammars designed on WikiSQL do not transfer to Spider's multi-table schemas.

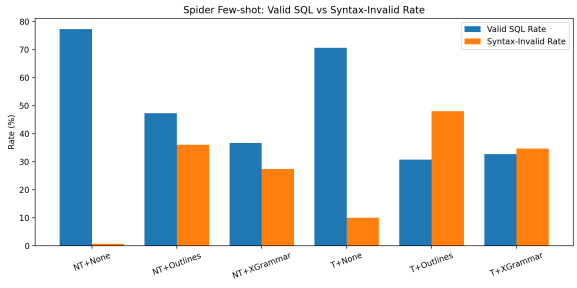


Figure 3: Spider few-shot: valid SQL rate versus syntax-invalid rate. Both constrained decoders reduce validity and increase malformed outputs, with the largest degradation under Outlines in thinking mode.

the example may even mislead the model when the test schema differs substantially.

Third, thinking mode retains a clear advantage under unconstrained decoding (36.0% vs. 23.3% EA in zero-shot), but this advantage disappears under constrained decoding, where both modes produce similarly low EA (7–11%). Constrained decoding erases the benefit of reasoning on hard benchmarks.

Spider error analysis. Figure 3 shows the complementary view of valid SQL rate versus syntax-invalid rate on Spider few-shot. Syntax-invalid outputs rise from 0.67% (unconstrained non-thinking) to 36.00% (Outlines) and 27.33% (XGrammar) in non-thinking mode, and from 10.00% to 48.00% and 34.67% in thinking mode. The zero-shot results show the same pattern: syntax-invalid outputs remain low for the unconstrained settings (3.33% non-thinking, 5.33% thinking) but increase sharply under Outlines (36.67%, 54.67%) and XGrammar (40.67%, 26.67%). Rather than eliminating malformed SQL, constrained decoding introduces new failure modes on Spider.

In few-shot non-thinking mode, Outlines causes 51 regressions from valid to invalid SQL (mostly syntax errors and empty outputs), while XGrammar causes 64 (dominated by schema errors, syntax errors, and overgeneration). In few-shot thinking mode, Outlines causes 64 regressions and XGrammar 60. Representative case studies illustrate these failures. For the question *"What are the first and last names, and middle initials of all customers who did not make any orders?"*, the non-thinking unconstrained model generates a valid SQL query, whereas Outlines collapses to an empty output. On the same example, XGrammar produces malformed SQL with repeated conjunctions, yielding an overgeneration failure and a syntax error near `AND`. In the thinking setting, Outlines occasionally repairs malformed reasoning traces: for the question *"Which locations have paintings created before 1885 or after 1930?"*, the unconstrained model outputs unfinished reasoning text that is not executable SQL, while Outlines produces the valid query `SELECT location FROM Paintings WHERE year < 1885 OR year > 1930`. These improvements are rare—only 3 cases improve from invalid to valid under Outlines—and are overwhelmed by the larger number of regressions.

5.3 Cross-dataset Comparison

Figure 4 juxtaposes execution accuracy across all 24 conditions. The asymmetry between datasets is the central finding of our evaluation. On WikiSQL, XGrammar matches or exceeds the unconstrained baseline in every setting, and the best overall result (68.3%) comes from thinking + few-shot + XGrammar. On Spider, the same grammar loses 15–29pp relative to unconstrained decoding regardless of mode or prompting. This confirms that constrained decoding is dataset-dependent: a grammar that stabilizes generation on a narrow query distribution can become counterproductive when transferred to a broader one without re-tuning. The result also validates the experimental design of §4.1, which deliberately tests transfer rather than per-dataset optimization.

5.4 Latency and Throughput

We ask a practical-deployment question: how long does it take to produce one SQL query at infer-

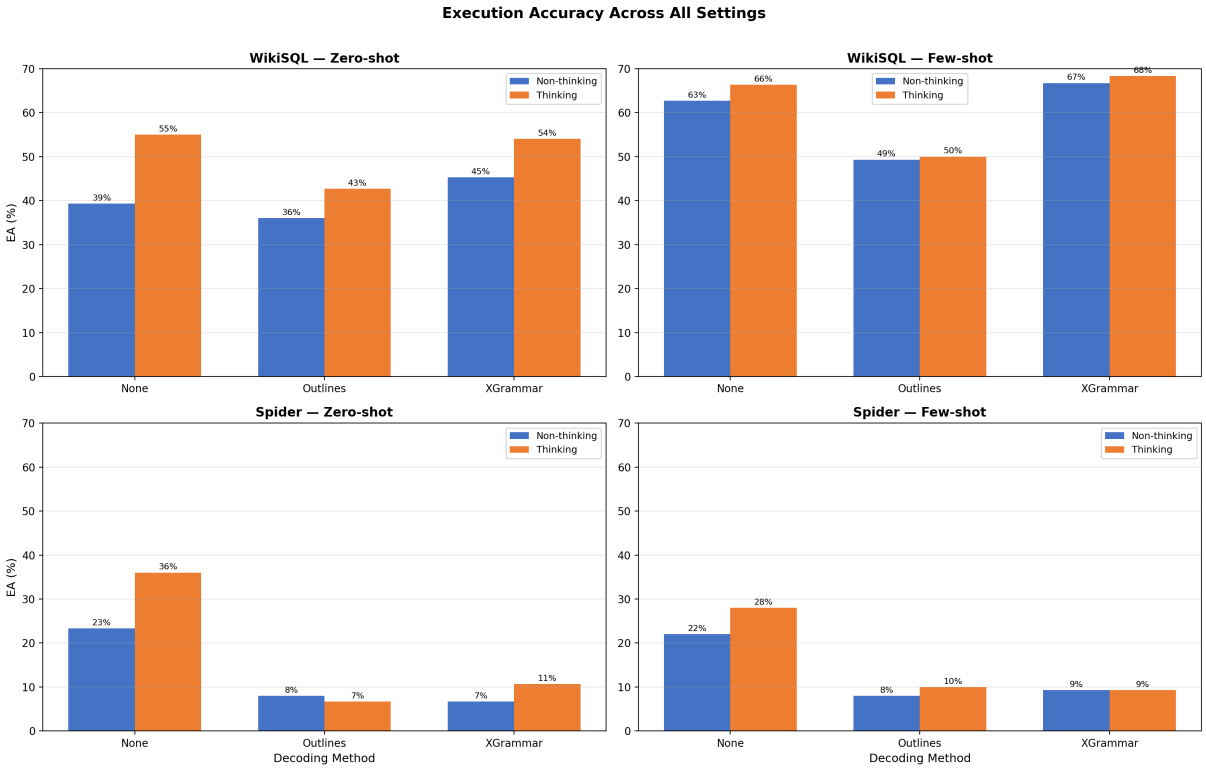


Figure 4: Execution accuracy across all 24 settings. WikiSQL (top) shows competitive constrained decoding, especially XGrammar under few-shot. Spider (bottom) shows consistent degradation under both constrained decoders regardless of prompting strategy.

ence time? Table 5 gives mean per-example generation latency, output token count, and throughput (tokens per second) across all 24 conditions. Within-dataset ratios between conditions are meaningful, but absolute latencies should not be compared across datasets (prompt length and token budget both differ).

Three patterns stand out. First, thinking mode dominates the wall-clock budget: non-thinking to thinking raises per-example latency by one to two orders of magnitude, and the blow-up tracks the generated token count almost one-for-one, so the extra cost comes from the `<think>` trace rather than from inference overhead. Second, XGrammar is essentially free per token, sitting within a few tok/s of the unconstrained baseline (59.2 vs. 67.9 tok/s on WikiSQL non-thinking zero-shot; 22.7 vs. 23.3 tok/s on Spider non-thinking zero-shot), consistent with its design goal (Dong et al., 2025). Outlines, by contrast, runs at roughly one quarter to one half of the baseline throughput (16.7 vs. 67.9 tok/s on WikiSQL; 11.9 vs. 23.3 tok/s on Spider), both because our runner is not batched and because its FSM wrapper is driven from Python at every step. Third, Outlines often shortens the output by forcing early termination once the regex reaches an accepting state ($60 \rightarrow 23$ tokens on WikiSQL non-thinking; $1178 \rightarrow 472$ on WikiSQL thinking), so its absolute wall-clock can undercut XGrammar even at lower tok/s. On Spider thinking, Outlines is in fact the fastest condition. For deployment, XGrammar pays the closest-to-zero latency tax, while Outlines trades per-token throughput for shorter outputs and should be evaluated jointly with the accuracy numbers in §5.

6 Conclusion

We studied constrained decoding as a purely inference-time intervention for NL2SQL using Qwen3-0.6B. We found that constrained decoding is dataset-dependent: XGrammar improves validity and execution accuracy on WikiSQL, where the grammar covers the query space well, but both XGrammar and Outlines degrade performance on Spider, where the same grammar transfers poorly to open multi-table schemas. The CFG–regex gap also matters in practice, as Outlines consistently trails XGrammar on WikiSQL due to the structural loss of projecting a CFG into a regular expression.

7 Limitations

Our study has several limitations. First, constrained decoding enforces structure but not semantics: a

Dataset	Shot	Mode	Latency (ms)			Tokens			Tok/s		
			None	Outl.	XGram.	None	Outl.	XGram.	None	Outl.	XGram.
WikiSQL	Zero	Non-think	**878**	1381	997	59.6	**23.0**	59.0	**67.9**	16.7	59.2
		Thinking	29235	**28927**	32771	1178.2	**471.9**	1261.6	**40.3**	16.3	38.5
	Few	Non-think	**1319**	1931	1795	44.4	**23.3**	59.7	**33.7**	12.1	33.2
		Thinking	**33897**	36066	34630	1095.1	**493.7**	1094.5	**32.3**	13.7	31.6
Spider	Zero	Non-think	**2420**	4248	3097	56.5	50.8	**70.4**	**23.3**	11.9	22.7
		Thinking	64132	**48948**	67546	952.5	**786.7**	991.7	14.9	**16.1**	14.7
	Few	Non-think	**2233**	3786	3230	44.0	47.3	**69.9**	19.7	12.5	**21.6**
		Thinking	59837	**52035**	68218	929.9	**830.1**	1016.2	15.5	**16.0**	14.9

Table 5: Latency, token count, and throughput reorganized by dataset, shot setting, and reasoning mode. **Bold highlights characteristic extremes per row**: unconstrained decoding (None) consistently achieves the fastest latency and highest throughput, while Outlines minimizes token count at a significant speed cost. XGrammar maintains token lengths similar to None with moderate overhead, representing a balanced efficiency–control trade-off.

query can be valid, executable, and still logically wrong. Second, we evaluate a single small backbone (Qwen3-0.6B) on an Apple M1 Pro with `mps`; absolute accuracies, and especially the thinking-mode vs. non-thinking gap, may look different on larger Qwen3 variants or on CUDA GPUs with different kernels. Third, we evaluate on fixed 300-example (WikiSQL) and 150-example (Spider) subsets, which gives tighter paired comparisons across conditions but wider confidence intervals than the full test sets. Fourth, we do not study fine-tuning or schema retrieval, which are known to improve both baselines and constrained decoders; our claims therefore concern decoding-time interventions only. Finally, we evaluate two constrained decoders (XGrammar, Outlines) chosen as representative CFG and regex implementations; the specific grammars we designed are not guaranteed to be optimal, and a more carefully tuned grammar could shift the Spider results.

References

Jinze Bai, Shuai Bai, Shusheng Yang, Shijie Wang, Sinan Tan, Peng Wang, Junyang Lin, Chang Zhou, and Jingren Zhou. 2023. Qwen-VL: A versatile vision-language model for understanding, localization, text reading, and beyond. *arXiv preprint arXiv:2308.12966*.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, and 1 others. 2020. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Yixin Dong, Charlie F Ruan, Yaxing Cai, Ziyi Xu, Yilong Zhao, Ruihang Lai, and Tianqi Chen. 2025. Xgrammar: Flexible and efficient structured generation engine for large language models. *Proceedings of Machine Learning and Systems*, 7.

dottxt-ai. 2025. Outlines: Structured text generation with llms. Project documentation.

Noam Gat. 2024. Lm format enforcer. GitHub repository.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*.

Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yutaka Iwasawa. 2022. Large language models are zero-shot reasoners. *arXiv preprint arXiv:2205.11916*.

Fangyu Lei, Jixuan Chen, Yuxiao Ye, Ruisheng Cao, Dongchan Shin, Hongjin Su, Zhaoqing Suo, Hongcheng Gao, Wenjing Hu, Pengcheng Yin, and 1 others. 2024. Spider 2.0: Evaluating language models on real-world enterprise text-to-sql workflows. *arXiv preprint arXiv:2411.07763*.

Jiachang Liu, Dinghan Shen, Yizhe Zhang, Bill Dolan, Lawrence Carin, and Weizhu Chen. 2022. What makes good in-context examples for GPT-3? In *Proceedings of Deep Learning Inside Out (DeeLIO 2022)*.

Qwen Team. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. 2021. Picard: Parsing incrementally for constrained auto-regressive decoding from language models. *arXiv preprint arXiv:2109.05093*.

Bailin Wang, Richard Shin, Xiaodong Liu, Oleksandr Polozov, and Matthew Richardson. 2020. Rat-sql: Relation-aware schema encoding and linking for text-to-sql parsers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7567–7578. Association for Computational Linguistics.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-thought prompting

elicits reasoning in large language models. *arXiv preprint arXiv:2201.11903*.

Brandon T. Willard and Rémi Louf. 2023. Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702*.

Xiaojun Xu, Chang Liu, and Dawn Song. 2017. Sqlnet: Generating structured queries from natural language without reinforcement learning. In *International Conference on Learning Representations (ICLR)*.

An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Ma, and 40 others. 2024. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*.

Tao Yu, Zifan Li, Zilin Zhang, Rui Zhang, and Dragomir Radev. 2018a. Typesql: Knowledge-based type-aware neural text-to-sql generation. In *Proceedings of NAACL-HLT 2018, Volume 2 (Short Papers)*, pages 588–594. Association for Computational Linguistics.

Tao Yu, Rui Zhang, Heyang Er, Suyi Li, Eric Xue, Bo Pang, Xi Victoria Lin, Yi Chern Tan, Tianze Shi, Zihan Li, Youxuan Jiang, Michihiro Yasunaga, Sungrok Shim, Tao Chen, Alexander Fabbri, Zifan Li, Luyao Chen, Yuwen Zhang, Shreya Dixit, and 5 others. 2019. Cosql: A conversational text-to-sql challenge towards cross-domain natural language interfaces to databases. In *Proceedings of EMNLP-IJCNLP 2019*, pages 1962–1979. Association for Computational Linguistics.

Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. 2018b. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. *arXiv preprint arXiv:1809.08887*.

Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, and Zheyan Luo. 2024. Llamafactory: Unified efficient fine-tuning of 100+ language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, pages 400–410. Association for Computational Linguistics.

Victor Zhong, Caiming Xiong, and Richard Socher. 2017. Seq2sql: Generating structured queries from natural language using reinforcement learning. *arXiv preprint arXiv:1709.00103*.

A Constrained decoding grammars

This appendix lists the exact regexes and EBNF grammars used in §4.5. The patterns below reflect the code in `src/inference/nltosql.py`: `build_wikisql_regex`, `build_spider_regex`, and `build_sql_ebnf`. Because the patterns exceed a single column's width, each grammar is displayed as a full-width figure spanning both columns.

B Prompt templates

System and user prompts are constructed as described in §4.4. The WikiSQL user prompt fixes the table name to `table` and lists eleven rules that enforce a single-column `SELECT`, lowercased string literals, and the canonical WikiSQL `SELECT . . . FROM table [WHERE . . . ]` shape. The Spider system prompt instructs the model to use only listed foreign keys for `JOIN`s and to qualify columns with their table name when the same column name occurs in multiple tables. In the few-shot regime, up to $N{=}1$ retrieved examples are prepended to the user turn: WikiSQL shots include their schema, Spider shots do not.

Figures 8 and 9 show the concrete rendered prompts for a WikiSQL and a Spider example, respectively, exactly as they are fed to the Qwen chat template (`system` + `user` turns; the shot block is inlined in the `user` turn).

C Post-processing examples

Table 6 illustrates each rule of the post-processing pipeline described in §4.5. Every row shows a raw model output on the left, the cleaned output actually scored by the evaluator on the right, and the rule that fired. Rows where the output is rejected map to an empty string and count as invalid SQL.

```
ident   = [A-Za-z_][A-Za-z0-9_ ]*
number  = -?\d+(?:\.\d+)?
string  = '[^']*'
value   = (?:string | number)
op      = (?:= | != | > | < | >= | <=)
agg     = (?: COUNT\(\*\) | COUNT\(ident\)
           | MAX\(ident\) | MIN\(ident\)
           | SUM\(ident\) | AVG\(ident\) )
sel     = (?:\* | ident | agg)
cond    = ident \s+ op \s+ value

sql     = SELECT \s+ sel \s+ FROM \s+ table
          (?: \s+ WHERE \s+ cond
              (?: \s+ (AND|OR) \s+ cond ){0,1}
          )? \s*;?
```

Figure 5: Outlines regex for WikiSQL. A single dataset-wide regex covers the entire query space.

```
table_union = t1 | t2 | ... | tn              # from schema
ident   = [A-Za-z_][A-Za-z0-9_ ]*
value   = quoted | number | bare
op      = (= | != | < | > | <= | >= | LIKE | IN | NOT IN)
sel     = (\* | ident | ident.ident | ident(ident))
cond    = ident(\.ident)? \s+ op \s+ value
join    = (\s+ JOIN \s+ (table_union) \s+ ON \s+ cond){0,3}
where   = (\s+ WHERE \s+ cond (\s+ (AND|OR) \s+ cond){0,3})?
order   = (\s+ ORDER \s+ BY \s+ ident(\.ident)? (\s+(ASC|DESC))?)?
limit   = (\s+ LIMIT \s+ [1-9][0-9]{0,5})?
sql     = SELECT \s+ (DISTINCT \s+)? sel \s+ FROM \s+
          (table_union) join where order limit \s*;?
```

Figure 6: Outlines regex for Spider, parameterized by the current database. The union of table names is inlined as a character class for each database.

```
root         ::= select_stmt                            # non-thinking
             ::= think_block select_stmt | select_stmt  # thinking
select_stmt  ::= "SELECT" ws select_expr ws "FROM" ws
                 table_name join_clause? where_clause
                 order_clause limit_clause
select_expr  ::= agg_expr | column_name | "*"
agg_expr     ::= agg_func ws? "(" ws? ("*" | column_name) ws? ")"
agg_func     ::= "COUNT" | "MAX" | "MIN" | "SUM" | "AVG"
where_clause ::= ""
             |  ws "WHERE" ws condition
             |  ws "WHERE" ws condition ws AND_OR ws condition
             |  ...  (up to max_conditions = 4)
condition    ::= column_name ws op ws value
op           ::= "=" | "!=" | ">" | "<" | ">=" | "<="
             |  "LIKE" | "IN" | "NOT" ws "IN"
             |  "IS" ws "NULL" | "IS" ws "NOT" ws "NULL"
value        ::= number | quoted_string | bare_word | "NULL"
order_clause ::= "" | ws "ORDER" ws "BY" ws column_name
                      (ws ("ASC"|"DESC"))?
limit_clause ::= "" | ws "LIMIT" ws [1-9][0-9]{0,5}
join_clause  ::= (ws "JOIN" ws table_name ws "ON" ws condition)*
think_block  ::= "<think>" think_text "</think>" ws_opt
think_text   ::= ([^<] | "<" [^/])*
table_name   ::= <literal alternatives from schema>
column_name  ::= <literal alternatives from schema>
```

Figure 7: Shared XGrammar EBNF skeleton. Dataset-specific switches: WikiSQL uses `AND_OR = "AND"`, `allow_join = false`, `table_name = "table"`. Spider uses `AND_OR = ("AND" | "OR")`, `allow_join = true`, `table_name` = all tables of the current database.

```
[system]
You are a precise text-to-SQL assistant.

[user]
Here are some examples:

Example 1:
Schema:
Table: table
Columns: Player (text), No. (real), Nationality (text), Position (text),
         Years in Toronto (text), School/Club Team (text)
Question: What school did player number 6 come from?
SQL: SELECT School/Club Team FROM table WHERE No. = 6

Now convert the following:

You are a text-to-SQL system.

Task:
Convert the question into exactly one SQL query for the given schema.

Rules:
1. Output exactly one SQL query and nothing else.
2. The query must start with SELECT.
3. Use only the table name: table
4. Use only columns that appear in the schema, using the exact column name
   as listed.
5. Always wrap string values in single quotes.
6. Lowercase all string values in WHERE conditions (e.g. 'john smith' not
   'John Smith').
7. Only add WHERE conditions for constraints explicitly stated in the
   question.
8. SELECT only the single column that directly answers the question --
   do not add extra columns.
9. Do not invent columns, values, or conditions not present in the
   question.
10. Do not add explanation.
11. Prefer the simplest correct WikiSQL-style query.

Instance ID: 1-10015132-11

Schema:
Table: table
Columns: Player (text), No. (real), Nationality (text), Position (text),
         Years in Toronto (text), School/Club Team (text)

Question:
How many schools did player number 3 attend?

SQL:
```

Figure 8: WikiSQL few-shot prompt as assembled in §4.4. The shot block is emitted inside the `user` turn so that both the demonstration schema and the target schema travel together.

```
[system]
You are working with the Spider dataset. Each database has multiple
tables. Queries may require JOINs across tables -- use only the foreign
key relationships listed in the schema. Queries range from simple
single-table lookups to multi-table JOINs with aggregation and ORDER BY.

Your task: convert the user's natural language question into exactly one
valid SQL query.

Rules:
1. Output ONLY the SQL query -- no explanation, no markdown, no code
   fences, nothing else.
2. The query must start with SELECT.
3. Use only tables and columns that appear in the schema.
4. Join tables only when necessary, and only through valid foreign key
   relationships shown in the schema.
5. Qualify column names with the table name (e.g. table.column) when the
   column exists in multiple tables.
6. Preserve string values exactly from the question when used in WHERE
   conditions.
7. Put text values in single quotes.
8. Do not invent columns, values, joins, or conditions not implied by
   the question.
9. Keep the query concise and correct for the schema.

[user]
Here are some examples:

Example 1:
Question: What are the names of all stadiums that did not have any
concert in 2014?
SQL: SELECT name FROM stadium WHERE stadium_id NOT IN
     (SELECT stadium_id FROM concert WHERE year = 2014)

Now convert the following:

Schema:
Tables: stadium, singer, concert, singer_in_concert
stadium(stadium_id, location, name, capacity, highest, lowest, average)
singer(singer_id, name, country, song_name, song_release_year, age,
       is_male)
concert(concert_id, concert_name, theme, stadium_id, year)
singer_in_concert(concert_id, singer_id)
Foreign keys: concert.stadium_id -> stadium.stadium_id,
              singer_in_concert.singer_id -> singer.singer_id,
              singer_in_concert.concert_id -> concert.concert_id

Question:
How many concerts were held in 2014 or 2015?

SQL:
```

Figure 9: Spider few-shot prompt. Shot schemas are omitted because the demonstration's database may differ from the target database; including them per shot would bloat the prompt without helping the model.

Raw model output	Cleaned output	Rule
`<think>We need the name.</think>SELECT name FROM table;`	`SELECT name FROM table`	Strip everything up to `</think>`, then truncate at the first semicolon.
`Let me reason. Finally SELECT id FROM table WHERE a = 1`	`SELECT id FROM table WHERE a = 1`	No `</think>` tag present, so fall back to the last occurrence of `select` and keep the tail.
“‘sql\nSELECT * FROM table\n“‘	`SELECT * FROM table`	Remove Markdown fences (“‘`sql` / “‘) and surrounding whitespace.
`<|im_start|>assistant\nSELECT a FROM table<|im_end|>`	`SELECT a FROM table`	Strip Qwen control markers (`<|im_start|>`, `<|im_end|>`, role tokens).
`SELECT name FROM table; SELECT age FROM table;`	`SELECT name FROM table`	Truncate at the first semicolon so the evaluator scores exactly one query.
`SELECT (a + b FROM table`	(empty)	Rejected by the unbalanced-parenthesis check.
`SELECT a FROM table WHERE b = 1 AND`	(empty)	Rejected by the trailing-operator check (query ends in a dangling `AND`).
`Here is the query: name FROM table`	(empty)	Rejected because no `SELECT` keyword is present after cleanup.

Table 6: Worked examples of the post-processing rules applied before evaluation. Rejected outputs become empty strings and are scored as invalid.